

Verifiable Multi-Dimensional Range Queries with M-HIBE Empty-Key Covers and ZK Accumulators

Draft notes for paper writing. This document reorganizes the old interactive M-HIBE idea from the PDFs into the current 3D non-interactive construction implemented in lang/go/mhibe/cmd_bench_3d_parallel.go.

1. Problem and Goal

We consider an outsourced database D containing tuples with d indexed attributes. A client issues a multi-dimensional range query

$$Q = [\ell_1, u_1] \times \cdots \times [\ell_d, u_d].$$

The server returns all tuples whose indexed attributes fall inside Q . The client must verify two properties:

1. Correctness/authenticity: every returned tuple is an authentic tuple from the outsourced dataset.
2. Completeness: no tuple satisfying Q has been omitted.

The zero-knowledge requirement is that the proof should not reveal database tuples outside the query answer, nor boundary records outside the query range.

The construction separates these two goals:

- Authenticity is handled by a ZK accumulator proof over the returned set.
- Completeness is handled by an M-HIBE empty-key cover: the server provides restricted keys for all empty regions inside the query range that are not occupied by returned answer points.

Intuitively, the returned tuples cover the non-empty points in the query box, and the M-HIBE empty keys cover the complement of those points inside the query box. If both covers verify and their union is exactly Q , the answer is complete.

2. From HIBE/WKD-IBE to M-HIBE

The original construction notes use the following observation.

In a one-dimensional HIBE tree, a key for a prefix node can derive keys for descendant nodes. A range is decomposed into canonical prefix nodes, so a key for an empty prefix certifies that every leaf below that prefix is empty.

For multiple dimensions, the scheme represents a multi-dimensional identity as a WKD-IBE pattern. For a d -dimensional database with ℓ bits per coordinate, the pattern length is $d\ell$. The first ℓ slots encode dimension 1, the next ℓ slots encode dimension 2, and so on. Unspecified suffix bits are wildcards.

For example, in 3D:

$$P = p_x \parallel p_y \parallel p_z.$$

is expanded to a fixed-length wildcard pattern:

$$P = p_x *^{\ell-|p_x|} \parallel p_y *^{\ell-|p_y|} \parallel p_z *^{\ell-|p_z|}.$$

A child pattern matches a parent pattern if every fixed bit in the parent is the same in the child. Therefore, a secret key for a parent empty region can derive a secret key for any smaller subregion.

This gives the key property used by the protocol:

If the data owner gives the server an M-HIBE key for an empty region R , then the server can derive keys for subregions of R , but cannot derive keys for regions not contained in R .

The construction notes call these keys "commitments". In the current paper draft, it is clearer to call them empty-region keys or empty-key commitments: they are cryptographic witnesses that a region was certified empty during preprocessing.

3. Original Interactive Scheme

The earliest scheme is naturally interactive.

1. The data owner preprocesses the dataset and gives the server M-HIBE secret keys for globally empty regions.
2. The client sends a range query Q .
3. The server returns an answer set A .
4. From Q and A , the client computes the empty regions it expects the server to prove. These regions are the portions of Q not occupied by answer points.
5. The client challenges the server on the canonical cover of those empty regions.
6. The server proves knowledge of, or reveals restricted versions of, the corresponding M-HIBE keys.
7. The client verifies that the returned tuples plus the proved empty regions exactly cover Q .

For 2D, the old notes describe this through regions of the form $x|y$. The data owner generates two kinds of commitments:

- $\text{Commit}(*|\perp)$: first-dimension empty regions where no point has an x coordinate under that prefix.
- $\text{Commit}(x|*)$: for each non-empty first-dimension prefix x , second-dimension empty ranges among the points whose first coordinate is under x .

The completeness proof says: for any empty challenge region $p|q$, either a first-dimension empty commitment covers it, or a commitment under a relevant first-dimension prefix covers it. This reduces storage from the trivial full grid construction to roughly $O(n\ell^2)$ keys for 2D.

The same idea generalizes to higher dimensions by fixing prefixes in earlier dimensions and storing empty covers in later dimensions.

4. Current 3D Construction: XY-Parent Supplement Cover

The current implemented construction specializes the above idea to 3D. The three dimensions in the TPC-H experiment are:

```
X = shipdate
Y = encoded discount
Z = quantity
```

Each coordinate is encoded into a fixed **BitLength**-bit binary string. In the current benchmark, **BitLength = 12**.

4.1 Preprocessing Index

The preprocessing builds two occupancy maps:

$$\text{XToY}[p_x] = \{y : \exists(x, y, z) \in D \text{ with } p_x \preceq x\}.$$

$$\text{XYToZ}[p_x, p_y] = \{z : \exists(x, y, z) \in D \text{ with } p_x \preceq x \text{ and } p_y \preceq y\}.$$

For every database point (x, y, z) , the owner inserts:

- y into $\text{XToY}[p_x]$ for every prefix $p_x \preceq x$.
- z into $\text{XYToZ}[p_x, p_y]$ for every pair of prefixes $p_x \preceq x$ and $p_y \preceq y$.

These maps allow the owner to find empty child ranges without enumerating the whole 3D universe.

4.2 Global Empty Parent Regions

Given a query box, the server touches only prefix nodes that intersect the query in the X and Y dimensions. For each touched prefix p_x , it produces:

1. Empty- Y regions:

$$p_x \parallel e_y \parallel *^\ell.$$

These certify that no point exists for that p_x prefix and e_y subrange, for any z .

2. Empty- Z regions under occupied (p_x, p_y) parents:

$$p_x \parallel p_y \parallel e_z.$$

These certify that, although the (x, y) parent has some points, the specified z subrange is empty.

The implementation then keeps maximal patterns, so redundant subregions are removed.

This is the "XY-parent supplement" cover: it does not attempt to precompute every 3D empty box. Instead, it stores/touches parent empty regions aligned with X and (X, Y) prefixes, and derives query-scoped children only when a query arrives.

5. Query-Time Proof Generation

At query time, the server performs the following steps.

5.1 Canonical Query Range Keys

The query range Q is decomposed into canonical prefix boxes. These are the normal range-cover keys for the query. In the reported experiment, there are 24 query canonical range keys.

These keys represent the query authorization material and are separate from the empty-region proof keys.

5.2 Query-Scoped Empty Patterns

The global empty parent regions may extend outside the query. Therefore the server intersects each global empty parent pattern with the query box.

For every non-empty intersection, it decomposes the intersection back into canonical prefix boxes and obtains query-scoped empty patterns. Then it runs a greedy set-cover minimization over the actual empty points inside Q to remove unnecessary overlaps.

In the 120K TPC-H run:

```
Global empty parent regions touched: 20985
Query-scoped empty regions selected: 4543
Empty spatial points covered: 24051
Matching rows: 2327
Unique matching spatial points: 2229
```

The query volume is therefore:

```
24051 empty points + 2229 occupied points = 26280 points.
```

5.3 Empty-Key Derivation

For every selected query-scoped empty pattern child, the server finds a global empty parent pattern parent such that:

$$\text{parent} \supseteq \text{child}.$$

Then it derives:

$$SK_{\text{child}} = \text{Derive}(SK_{\text{parent}}, \text{child}).$$

In the paper-level protocol, SK_{parent} is produced by the data owner during preprocessing/upload and stored by the server. The server should not possess the master secret key.

Implementation note: the current benchmark derives the touched parent keys from the master key inside the benchmark program to simulate the offline materialized parent keys and measure the amount of parent key material touched. The paper should describe the real deployment as:

$$\text{Owner} : \text{MSK} \rightarrow \{SK_{\text{parent}}\} \rightarrow \text{Server}.$$

$$\text{Server} : \{SK_{\text{parent}}\} \rightarrow \{SK_{\text{child}}\}.$$

not as:

$$\text{Server} : \text{MSK} \rightarrow \{SK_{\text{child}}\}.$$

5.4 Parallelization

The parallel implementation does not change the mathematical proof. It only parallelizes independent key operations:

- finding the parent for each scoped pattern;
- generating unique parent keys;
- deriving child keys grouped by parent;
- sampling key-decryption checks during verification.

With one worker, the parallel program falls back to the serial time. With 20 workers, the 120K TPC-H run reduces WKD-IBE delegation time from about 68.7 s to about 7.4 s, while producing the same number of regions and the same key material sizes.

For the paper, report both:

- serial implementation as an ablation/baseline;
- parallel implementation as the optimized system result.

6. What the Key Verification Does

This is the part that is easy to forget because there are two different "verification" ideas.

6.1 Cryptographic Meaning of an Empty Key

An M-HIBE key for pattern P is valid only if it was generated from the master key or derived from a valid ancestor key. Under the security of WKD-IBE/M-HIBE, a malicious server that does not have an authorized ancestor key should not be able to forge a valid key for an arbitrary pattern.

Therefore, if the verifier receives a valid key for an empty pattern P , it treats this as evidence that the data owner certified P as an empty region or certified an ancestor empty region from which P can be derived.

The server may reveal these query-scoped empty keys because they correspond only to empty regions inside the client's query. They do not decrypt database records outside the query answer.

6.2 Same-Pattern Decryption Check

In the prototype, a supplied derived key is checked as follows:

1. Convert the claimed pattern P into a WKD-IBE attribute list.
2. Sample a random message m .
3. Encrypt m under the same attribute list P .
4. Decrypt the ciphertext using the supplied key SK_P .
5. Accept this key sample if the decrypted message equals m .

In pseudocode:

```
CheckKey(params,  $P$ ,  $SK_P$ ) :  attrs  $\leftarrow$  PatternToAttributes( $P$ ),  
                                $m \leftarrow$  RandomMessage(),  
                                $ct \leftarrow$  Encrypt(params, attrs,  $m$ ),  
                                $m' \leftarrow$  Decrypt( $ct$ ,  $SK_P$ ),  
                               return  $m' = m$ .
```

This verifies that the supplied key is functionally a valid decryption key for the claimed pattern. A malformed or unrelated key will fail except with negligible probability.

6.3 Sampling vs Full Verification

The current benchmark samples at most 64 derived empty keys:

```
MaxKeyPatternDecryptChecks = 64
```

Thus the printed line:

```
64 sampled verification keys successfully decrypted same-pattern WKD-IBE ciphertexts.
```

means "64 sampled keys passed the functional decryption sanity check." It does not mean the implementation decrypted and checked all 4543 selected empty keys.

For the paper, use precise language:

- The coverage check is exhaustive over the query grid in the experiment.
- The key-decryption check is sampled in the benchmark to keep timing stable.
- A full-check mode can verify every provided empty key at cost linear in the number of empty-region keys.

Recommended paper wording:

For performance reporting, we sample up to 64 returned empty-region keys and verify each sampled key by encrypting a fresh random message under the claimed pattern and decrypting it with the supplied key. This test validates the correctness of the derived WKD-IBE key material. The exhaustive part of the client completeness check is the geometric coverage test, which verifies that the selected empty patterns cover exactly the query points not occupied by the returned answer. A full key audit can check all returned keys with linear additional cost.

7. Client Completeness Check

The client verifies two things about the empty-region patterns.

7.1 Geometry

For every empty pattern, the verifier converts the pattern into coordinate bounds and checks:

1. The bounds are inside the query range.
2. The pattern does not cover any returned answer point.
3. Every query point not occupied by a returned answer point is covered by at least one empty pattern.

Let:

$$V_Q = \{\text{all grid points inside } Q\}.$$

$$A_Q = \{\text{unique spatial points in the returned answer}\}.$$

$$E_Q = \{\text{points covered by empty patterns}\}.$$

The completeness condition is:

$$A_Q \cup E_Q = V_Q, \quad A_Q \cap E_Q = \emptyset.$$

The implementation reports this as:

```
coveredEmptyPoints + realSpatialVolume == totalQueryVolume
```

In the benchmark, `realSpatialVolume` is computed from the true matching rows because the benchmark has the dataset in memory. In the protocol, this set is the set of unique spatial points in the server's returned answer. Authenticity of the returned answer is checked separately by the accumulator proof.

7.2 Key Validity

The verifier checks that the supplied key material matches the claimed patterns using the same-pattern decryption test above. In a full verification mode, this is done for every empty-region key. In the reported benchmark, it is sampled.

Together, geometry plus key validity implies:

- If the server omits a real answer point, the omitted point would have to be covered by an empty-region key.
- But the server should not possess a valid empty-region key covering a real database point, because the data owner only issued empty-region keys.
- Therefore a cheating server cannot both omit a real point and pass the empty cover verification, except by forging an M-HIBE key or breaking the data owner's preprocessing assumption.

8. Authenticity via ZK Accumulator

The M-HIBE empty-key cover proves completeness conditioned on the returned answer points. We still need to prove that the returned tuples are authentic members of the outsourced dataset.

The implementation uses a bilinear/polynomial accumulator layer.

Let:

$$D = \{\text{all encoded database points}\}, \quad I = \{\text{encoded returned/query-matching points}\}, \quad X = D \setminus I$$

The server computes commitments/digests to D and X , and commits to the polynomial representing I . It then produces:

- a ZK membership-style proof tying the answer commitment to the database digest and complement digest;
- a ZK degree check proof for the answer polynomial.

In the prototype, this corresponds to:

$$\begin{aligned} \text{digest}_D &\leftarrow \text{Commit}(D), & \text{digest}_X &\leftarrow \text{Commit}(X). \\ f_I &\leftarrow \text{PolyTree}(I), & C_I &\leftarrow \text{PedersenCommit}(f_I). \\ \pi_{\text{mem}} &\leftarrow \text{ZKMemProver}(C_I, \text{digest}_X, \text{transcript}). \\ \pi_{\text{deg}} &\leftarrow \text{ZKDegCheckProver}(C_I, f_I, H(\pi_{\text{mem}}, \text{transcript})). \end{aligned}$$

The verifier checks:

$$\begin{aligned} \text{ZKMemVerifier}(\pi_{\text{mem}}, \text{digest}_D, C_I, \text{transcript}) &= 1. \\ \text{ZKDegCheckVerifier}(C_I, \pi_{\text{deg}}, H(\pi_{\text{mem}}, \text{transcript})) &= 1. \end{aligned}$$

This layer should be written as a non-interactive ZK proof in the random-oracle model. The challenge values are derived from a transcript hash containing the public statement, query, answer commitment, accumulator digests, and previous proof messages.

Important writing point: do not describe this as the verifier sending random challenges online. The non-interactive version uses Fiat-Shamir-style challenge derivation:

$$\rho = H(\text{pp}, Q, C_A, \text{digest}_D, \text{digest}_X, \text{proof_prefix}).$$

The implementation's accumulator library already exposes prover/verifier APIs that take a transcript. In the final paper, define explicitly what is included in the transcript so that the proof is clearly non-interactive.

9. From Interactive to Non-Interactive

The original protocol was interactive because the client computed challenge regions after seeing the answer and asked the server to prove those regions.

The non-interactive transformation has two parts.

9.1 Deterministic Empty-Challenge Generation

The set of empty regions is no longer chosen by an online verifier message. It is deterministically derived from public/query-visible information:

$$\mathcal{E}_Q = \text{Cover}(Q, A_Q).$$

In the current optimized construction, the server derives this set by intersecting global empty parent regions with the query and then minimizing the query-scoped cover. The verifier can recompute or check the resulting geometry against the returned answer.

Because the challenge set is deterministic, the server can send in one message:

$$A, \mathcal{E}_Q, \{SK_E\}_{E \in \mathcal{E}_Q}, \pi_{\text{acc}}.$$

There is no need for a second client challenge round.

9.2 Fiat-Shamir for ZK Proofs

Any interactive accumulator subprotocol is made non-interactive by replacing verifier randomness with transcript-derived challenges.

The transcript should bind:

- public parameters;
- dataset digest;
- query bounds;
- returned answer commitment;
- empty-pattern list or its hash;
- accumulator proof messages generated so far.

This prevents the server from reusing a proof for a different query or a different answer.

The resulting protocol is one-round from the server to the client after the client sends the query:

$$\text{Client} \rightarrow \text{Server} : Q.$$

$$\text{Server} \rightarrow \text{Client} : (A, \mathcal{E}_Q, \{SK_E\}_{E \in \mathcal{E}_Q}, \pi_{\text{acc}}).$$

$$\text{Client} : \text{Verify}(Q, A, \mathcal{E}_Q, \{SK_E\}_{E \in \mathcal{E}_Q}, \pi_{\text{acc}}).$$

This is non-interactive in the proof-system sense: after the query is fixed, the verifier sends no random challenge messages.

10. Suggested Protocol Algorithms

Setup

Input: security parameter, dimension d , bit length ℓ .

1. Generate M-HIBE/WKD-IBE public parameters and master secret key.
2. Generate accumulator public parameters.
3. Publish public parameters.

Upload/Preprocessing

Input: dataset D , master secret key.

1. Encode each indexed tuple as a $d\ell$ -slot pattern or as a field element for the accumulator.
2. Build prefix occupancy indexes.
3. Generate global empty parent regions.
4. For each global empty parent region R , generate an M-HIBE key SK_R .
5. Send the outsourced dataset, accumulator commitment material, and empty parent keys to the server.
6. The data owner keeps or destroys the master secret key according to the deployment model; the server must not receive it.

Query/Prove

Input: query Q .

1. Server computes answer A .
2. Server computes query-scoped empty patterns from global empty parents.
3. Server derives restricted empty keys for the selected query-scoped patterns.
4. Server computes the ZK accumulator proof for answer authenticity.
5. Server sends $(A, \mathcal{E}_Q, \{SK_E\}_{E \in \mathcal{E}_Q}, \pi_{\text{acc}})$ to the client.

Verify

Input: query Q , answer A , empty patterns and keys, accumulator proof.

1. Verify the accumulator proof for A .
2. Deduplicate answer spatial points to obtain A_Q .
3. Verify every empty pattern lies inside Q .
4. Verify no empty pattern covers any point in A_Q .
5. Verify every grid point in $Q \setminus A_Q$ is covered by at least one empty pattern.
6. Verify the supplied empty keys match their claimed patterns. In the full protocol this can be all keys; in the benchmark it is sampled.
7. Accept iff all checks pass.

11. How to Present the Experimental Results

Use the parallel implementation as the main system result, but include the serial implementation as an ablation.

For the 120K TPC-H Q6-like query:

```
Serial 3D:
  Engine A total: 77.62 s
  WKD-IBE delegation: 68.78 s
  Client completeness check: 1.65 s
  Total server proving: 100.96 s
```

```
Parallel 3D, 20 workers:
  Engine A total: 16.32 s
  WKD-IBE delegation: 7.35 s
  Client completeness check: 0.16 s
  Total server proving: 39.96 s
```

Both runs produce the same logical proof shape:

```
Query canonical range keys: 24
Global empty parent regions: 20985
Query-scoped empty regions: 4543
Query range key material: 14.27 KB
Global parent key material touched: 1737.30 KB
Query-scoped empty key material: 1553.73 KB
ZK proof size: 1.20 KB
```

Recommended paper wording:

```
Parallelization affects only independent M-HIBE key derivations and verifier sampling checks; it does not change the selected empty-region cover or the proof material. We therefore report the parallel wall-clock time as the optimized implementation result and the serial time as an ablation isolating the effect of multi-core key derivation.
```

Also include the worker count and hardware:

```
The parallel M-HIBE prover uses 20 worker goroutines. Reported times are wall-clock times.
```

12. Caveats to Resolve Before Final Paper

1. The paper protocol should say the server stores owner-generated global empty parent keys. The prototype currently simulates this by deriving touched parent keys from the master key during benchmarking.
2. The benchmark samples at most 64 empty keys for same-pattern decryption checks. Either add a full-check experiment or clearly label this as a sampled key sanity check.
3. The final non-interactive ZK section should explicitly define the Fiat- Shamir transcript.

4. The verifier in the benchmark runs in a single combined program with the dataset available. In the paper, phrase the verifier's occupied points as the unique points in the returned answer, with authenticity supplied by the accumulator proof.
5. Decide whether the proof object reveals empty-region keys directly or uses a proof of knowledge of those keys. The current implementation reveals restricted query-scoped keys and checks them by test encryption/decryption. This is simpler and matches the benchmark, but the leakage statement should be written carefully.

13. Possible Section Skeleton for the Paper

1. Introduction
2. Background
 - Canonical range covers
 - WKD-IBE and M-HIBE
 - Polynomial/ZK accumulators
3. Problem Definition
 - System model
 - Correctness, completeness, zero-knowledge
4. Construction Overview
 - Authenticity layer
 - Completeness layer
5. M-HIBE Empty-Key Cover
 - Original interactive challenge view
 - 3D XY-parent supplement cover
 - Key derivation and verification
6. Non-Interactive Proof
 - Deterministic challenge generation
 - Fiat-Shamir accumulator transcript
7. Security Argument
 - Authenticity
 - Completeness
 - Zero-knowledge leakage discussion
8. Implementation
 - Serial and parallel M-HIBE derivation
 - Prototype caveats
9. Evaluation
 - TPC-H setup
 - Key material and proof size
 - Serial vs parallel ablation
10. Conclusion